

Task 1:

What is a thread and a Process?

A process is an instance of a program that is executing.

Has its own memory space

Contains code, data, and system resources (files, sockets, etc.)

Each process runs independently

Example: Opening Chrome, Word, or a game — each runs as its own process

Thread

A thread is the smallest unit of execution inside a process.

A process can have multiple threads

All threads in a process share the same memory

Threads are used for concurrent or parallel execution

Example: In a web browser (one process), one thread loads images, another runs scripts, another responds to clicks

Task 2:

Understand the below code and run it to see the output.. Need to explain...

8 min 10.42 to 10.50

```
class RunnableDemo implements Runnable {  
    private Thread t;  
    private String threadName;  
    RunnableDemo( String name){  
        threadName = name;  
        System.out.println("Creating " + threadName );  
    }  
    public void run() {  
        System.out.println("Running " + threadName );  
        try {  
            for(int i = 4; i > 0; i--) {
```

```

System.out.println("Thread: " + threadName + ", " + i);
// Let the thread sleep for a while.
Thread.sleep(50);
}
} catch (InterruptedException e) {
System.out.println("Thread " + threadName + " interrupted.");
}
System.out.println("Thread " + threadName + " exiting.");

}
public void start ()
{
System.out.println("Starting " + threadName );
if (t == null)
{
t = new Thread (this, threadName);
t.start ();
}
}
}
public class TestThread {
public static void main(String args[]) {
RunnableDemo R1 = new RunnableDemo( "Thread-1");
R1.start();
RunnableDemo R2 = new RunnableDemo( "Thread-2");
R2.start();
}
}
}

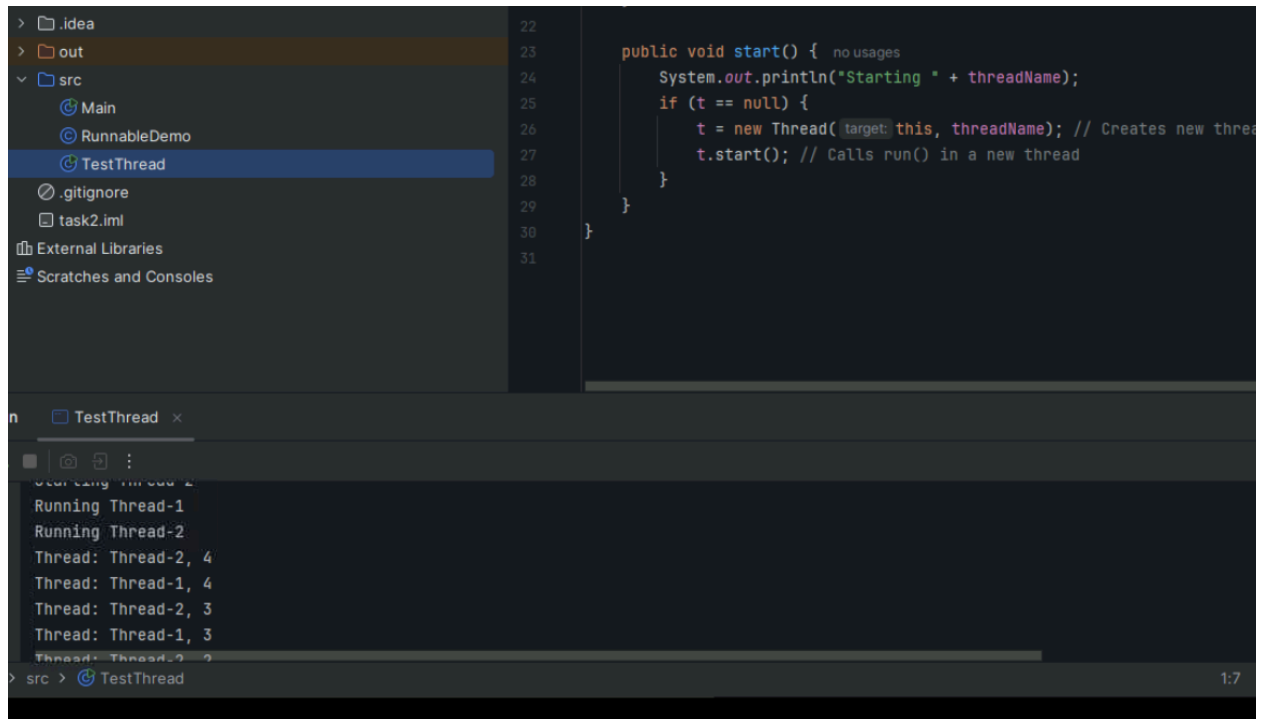
```

```
class RunnableDemo implements Runnable { no usages
    private Thread t; 3 usages
    RunnableDemo(String name) { 8 usages
        threadName = name;
        System.out.println("Creating " + threadName);
    }

    public void run() {
        System.out.println("Running " + threadName);
        try {
            for (int i = 4; i > 0; i--) {
                System.out.println("Thread: " + threadName + ", " + i);
                Thread.sleep(50); // Pauses thread for 50ms
            }
        }
    }
}
```

```
class RunnableDemo implements Runnable { no usages
    public void run() {
        Thread.sleep(50); // Pauses thread for 50ms
    }
    catch (InterruptedException e) {
        System.out.println("Thread " + threadName + " interrupted.");
    }
    System.out.println("Thread " + threadName + " exiting.");
}

    public void start() { no usages
        System.out.println("Starting " + threadName);
        if (t == null) {
            t = new Thread(target: this, threadName); // Creates new thread with
            t.start(); // Calls run() in a new thread
        }
    }
}
```



Task 3



In the above code ... try extending Thread class... and observe the output..

11.16 11.18

Understand:

ex:

Public Class1 extends Thread{ }

Or

4public Class1 extends Class2 implements Runnable ()

```

class MyThread extends Thread { 4 usages
    private String threadName; 7 usages

    MyThread(String name) { 2 usages
        threadName = name;
        System.out.println("Creating " + threadName);
    }

    public void run() {
        System.out.println("Running " + threadName);
        try {
            for (int i = 4; i > 0; i--) {
                System.out.println("Thread: " + threadName + ", " + i);
                Thread.sleep(millis: 50);
            }
        } catch (InterruptedException e) {

```

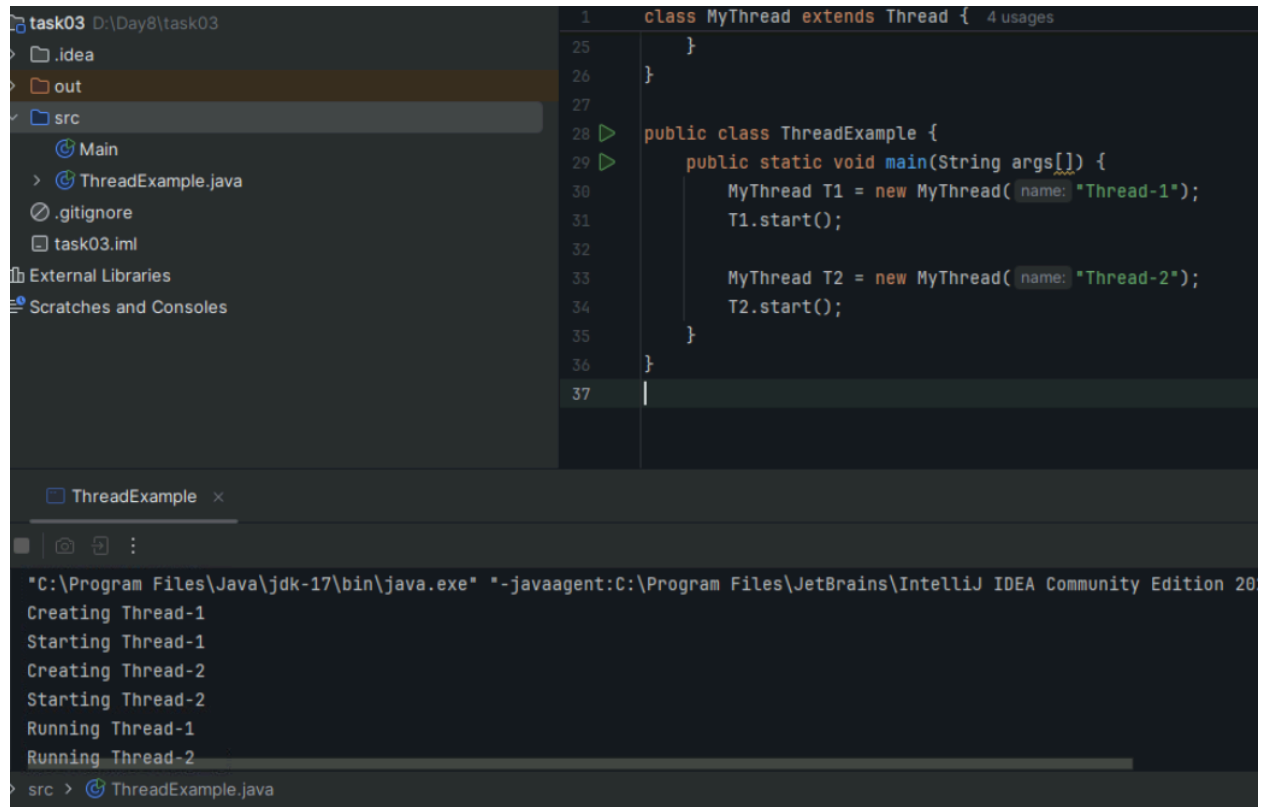
```

class MyThread extends Thread { 4 usages
    public void run() {
        }
    } catch (InterruptedException e) {
        System.out.println("Thread " + threadName + " interrupted.");
    }
    System.out.println("Thread " + threadName + " exiting.");
}

public void start() {
    System.out.println("Starting " + threadName);
    super.start(); // Must call super.start() to actually launch the thread
}
}

> public class ThreadExample {

```



Task 5:

Use synchronized method:

11.54 to 11.59 5 min

Hint:

1. Synchronized Method:

Synchronize the entire method to ensure only one thread can execute it at a time.

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

```

class Counter { 4 usages
    private int count = 0; 2 usages

    public synchronized void increment() { 1 usage
        count++;
    }

    public int getCount() { 1 usage
        return count;
    }
}

class ThreadDemo extends Thread { 4 usages
    private final Counter counter; 2 usages

    ThreadDemo(Counter counter) { 2 usages
        ...
    }
}

```

The screenshot shows the IntelliJ IDEA IDE. On the left, the Project Explorer displays the file structure with 'ThreadSyncDemo.java' selected. The main editor window shows the code for 'ThreadDemo' class, which extends 'Thread'. The code includes a 'private final Counter counter' field, a constructor 'ThreadDemo(Counter counter)' that initializes 'this.counter = counter;', and a 'run()' method. The 'run()' method contains a loop from 0 to 1000, calling 'counter.increment()' with a comment '// Thread-safe call'. The bottom status bar shows the command executed: '"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=6313...' and the output: 'Final count: 2000' and 'Process finished with exit code 0'.

```

13
14
15 class ThreadDemo extends Thread { 4 usages
16     private final Counter counter; 2 usages
17
18     ThreadDemo(Counter counter) { 2 usages
19         this.counter = counter;
20     }
21
22     public void run() {
23         for (int i = 0; i < 1000; i++) {
24             counter.increment(); // Thread-safe call
25         }
26     }
27 }
28

```

ThreadSyncDemo x

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=6313..."
 Final count: 2000
 Process finished with exit code 0

Task 6:

Using Sync Block

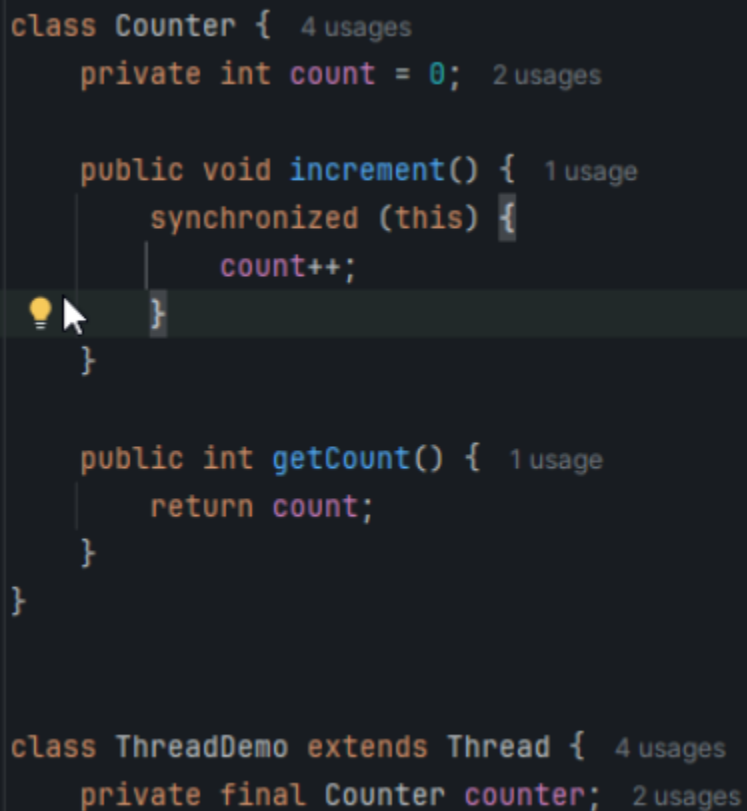
12.00 to 12.05

Hint:

2. Synchronized Block:

Synchronize a block of code instead of the entire method, providing more control and efficiency.

```
class Counter {  
    private int count = 0;  
  
    public void increment() {  
        synchronized (this) {  
            count++;  
        }  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```



A screenshot of a code editor showing Java code. The code defines a `Counter` class and a `ThreadDemo` class. The `Counter` class has a private `int count = 0`, a `public void increment()` method with a `synchronized (this)` block, and a `public int getCount()` method. The `ThreadDemo` class extends `Thread` and has a private final `Counter counter`. The code is color-coded, and usage counts are shown in the right margin: `Counter` (4 usages), `private int count = 0;` (2 usages), `public void increment() {` (1 usage), `synchronized (this) {` (1 usage), `count++;` (1 usage), `public int getCount() {` (1 usage), `return count;` (1 usage), `class ThreadDemo extends Thread {` (4 usages), and `private final Counter counter;` (2 usages). A mouse cursor is pointing at the closing brace of the `increment()` method.

```
class Counter { 4 usages  
    private int count = 0; 2 usages  
  
    public void increment() { 1 usage  
        synchronized (this) {  
            count++;  
        }  
    }  
  
    public int getCount() { 1 usage  
        return count;  
    }  
}  
  
class ThreadDemo extends Thread { 4 usages  
    private final Counter counter; 2 usages
```

```
class ThreadDemo extends Thread { 4 usages
    private final Counter counter; 2 usages

    ThreadDemo(Counter counter) { 2 usages
        this.counter = counter;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
    }
}
```

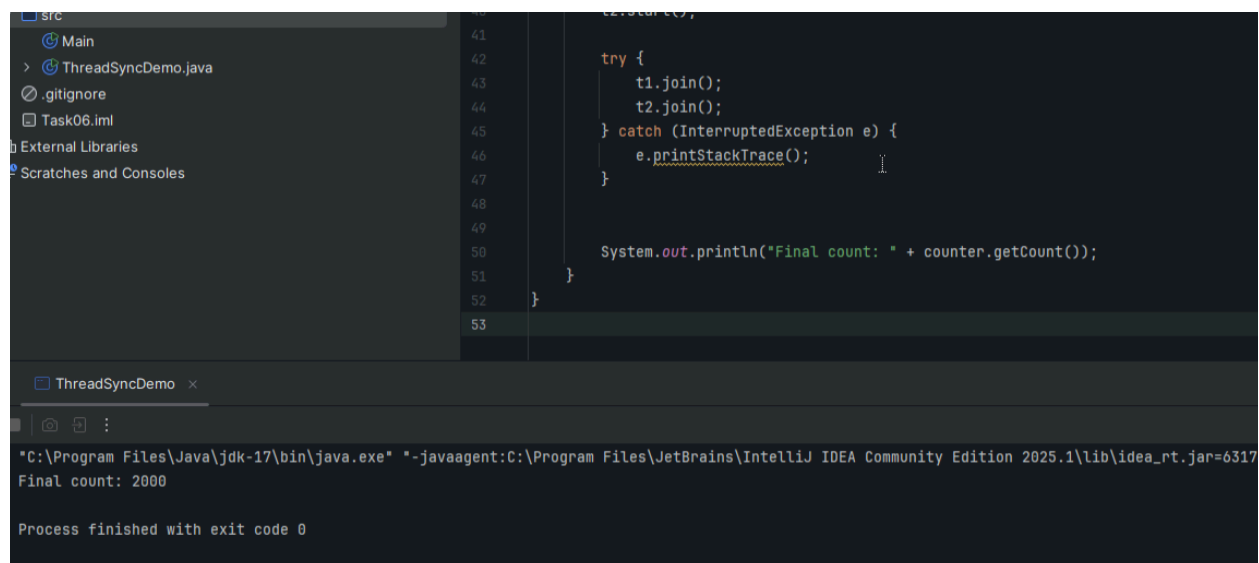
```
public class ThreadSyncDemo {
    public static void main(String[] args) {
        Counter counter = new Counter();
    }
}
```

```
public class ThreadSyncDemo {
    public static void main(String[] args) {
        Counter counter = new Counter();

        ThreadDemo t1 = new ThreadDemo(counter);
        ThreadDemo t2 = new ThreadDemo(counter);

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```



Task 7:

Using Static Sync

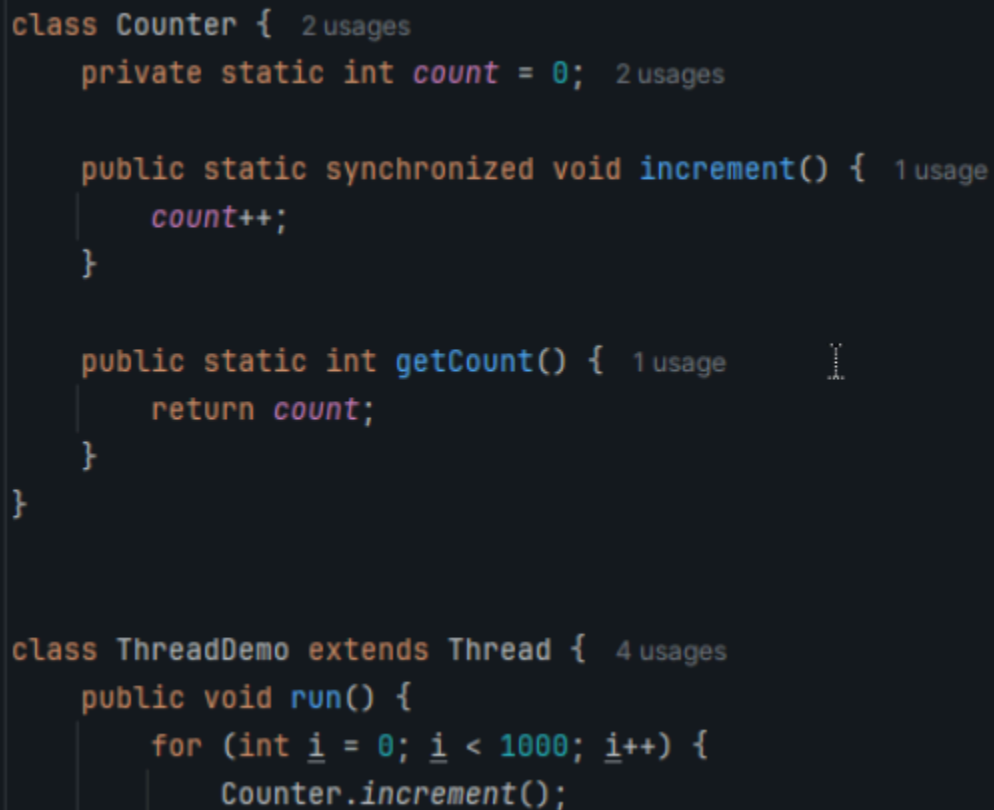
12.06 to 12.10

Hint:

3. Static Synchronization:

Synchronize static methods to ensure only one thread can execute them for the class, not the instance.

```
class Counter {  
    private static int count = 0;  
  
    public static synchronized void increment() {  
        count++;  
    }  
  
    public static int getCount() {  
        return count;  
    }  
}
```

A screenshot of a code editor showing the same Java code as above, but with IDE annotations. The code is color-coded, and each line or block is followed by a comment indicating the number of usages. For example, 'class Counter {' is followed by '2 usages', 'private static int count = 0;' by '2 usages', 'public static synchronized void increment()' by '1 usage', and so on. The code is displayed on a dark background with light-colored text.

```
class Counter { 2 usages  
    private static int count = 0; 2 usages  
  
    public static synchronized void increment() { 1 usage  
        count++;  
    }  
  
    public static int getCount() { 1 usage  
        return count;  
    }  
}  
  
class ThreadDemo extends Thread { 4 usages  
    public void run() {  
        for (int i = 0; i < 1000; i++) {  
            Counter.increment();  
        }  
    }  
}
```

```

class ThreadDemo extends Thread { 4 usages
    public void run() {
        for (int i = 0; i < 1000; i++) {
            Counter.increment();
        }
    }
}

public class ThreadSyncDemo {
    public static void main(String[] args) {
        ThreadDemo t1 = new ThreadDemo();
        ThreadDemo t2 = new ThreadDemo();

        t1.start();
        t2.start();
    }
}

```

The screenshot shows the IntelliJ IDEA IDE. The top pane displays the source code for `ThreadSyncDemo.java`, which includes the `ThreadDemo` class and the `ThreadSyncDemo` class. The `ThreadSyncDemo` class has a `main` method that creates two `ThreadDemo` objects, `t1` and `t2`, and starts them. The bottom pane shows the output of the program, which prints the final count of the `Counter` object, which is 2000.

```

31
32
33
34
35
36
37
38
39
40
41
42
43

```

```

try {
    t1.join();
    t2.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}

System.out.println("Final count: " + Counter.getCount());
}

```

ThreadSyncDemo x

```

C:\Program Files\Java\jdk-17\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=63212" -Dfile.encoding=UTF-8 -class
nal count: 2000

Process finished with exit code 0

```

Activate W

Task 8:

Using Locks

12.11 to 12.15

Hint:

4. Locks:

Use `java.util.concurrent.locks.Lock` for more sophisticated thread synchronization.

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class Counter {
    private int count = 0;
    private final Lock lock = new ReentrantLock();

    public void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock();
        }
    }

    public int getCount() {
        return count;
    }
}
```

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

// Counter class using ReentrantLock
class Counter { 4 usages
    private int count = 0; 2 usages
    private final Lock lock = new ReentrantLock(); 2 usages

    public void increment() { 1 usage
        lock.lock(); // Acquire the lock
        try {
            count++;
        } finally {
            lock.unlock(); // Always release the lock in finally block
        }
    }

    public int getCount() { 1 usage
```

```
    public int getCount() { 1 usage
    |     return count;
    }
}

// Thread class that works with the Counter
class ThreadDemo extends Thread { 4 usages
    private final Counter counter; 2 usages

    ThreadDemo(Counter counter) { 2 usages
    |     this.counter = counter;
    }

    public void run() {
```

```
// Thread class that works with the counter
class ThreadDemo extends Thread { 4 usages
    private final Counter counter; 2 usages

    ThreadDemo(Counter counter) { 2 usages
        this.counter = counter;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            counter.increment();
        }
    }
}

// Main class to run threads
public class ThreadSyncDemo {
    public static void main(String[] args) {
```

```

public class ThreadSyncDemo {
    public static void main(String[] args) {
        Counter counter = new Counter(); // shared object

        ThreadDemo t1 = new ThreadDemo(counter);
        ThreadDemo t2 = new ThreadDemo(counter);

        t1.start();
        t2.start();

        try {
            t1.join(); // wait for t1 to finish
            t2.join(); // wait for t2 to finish
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Final count: " + counter.getCount());
    }
}

```

The screenshot shows an IDE with the following components:

- Editor:** Displays the code for `ThreadSyncDemo.java`, lines 51 to 59. The code is identical to the one in the first block.
- Run Console:** Shows the output of the program:


```

      C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=63263" -Dfile.encoding=UTF-8 -classpath D:\Day8\ThreadS
      Final count: 2000
      Process finished with exit code 0
      
```

Task 10:

Dead Lock 👍

Example of Deadlock

```

class Resource {
    synchronized void method1(Resource r) {
        System.out.println(Thread.currentThread().getName() + " is executing

```

```

method1");
    try { Thread.sleep(100); } catch (InterruptedException e) {}
    r.method2(this);
}

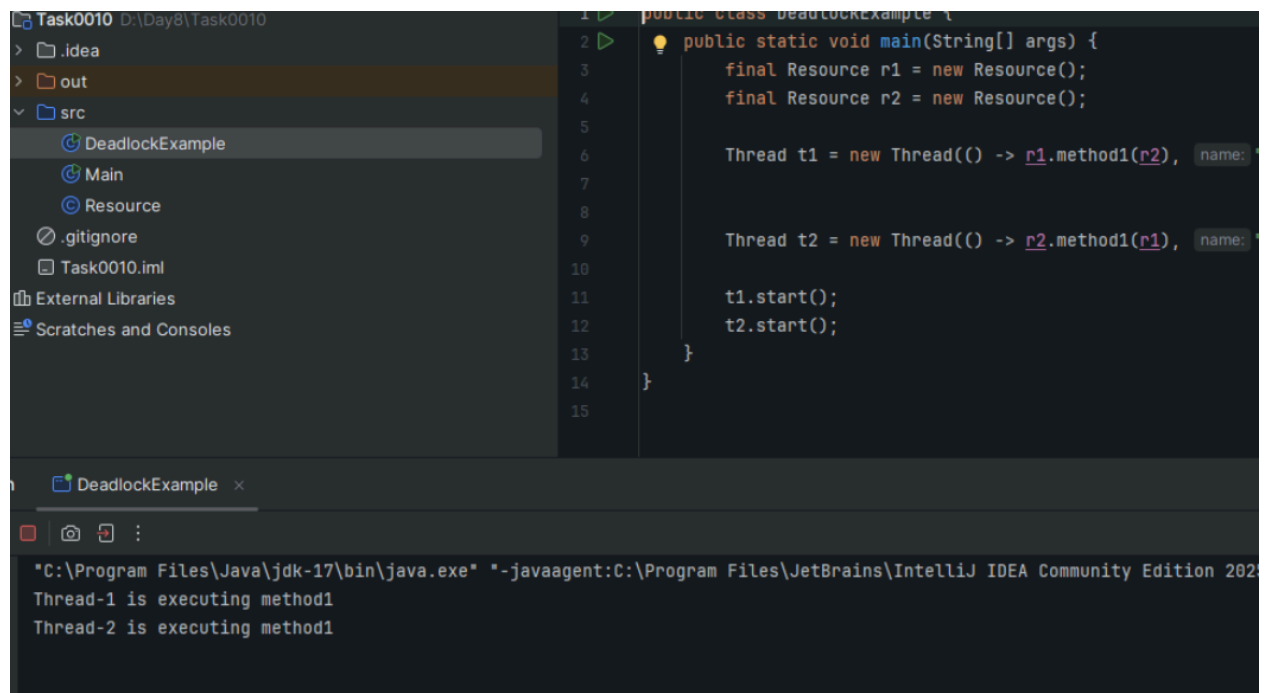
synchronized void method2(Resource r) {
    System.out.println(Thread.currentThread().getName() + " is executing
method2");
    try { Thread.sleep(100); } catch (InterruptedException e) {}
    r.method1(this);
}
}

public class DeadlockExample {
    public static void main(String[] args) {
        final Resource r1 = new Resource();
        final Resource r2 = new Resource();

        Thread t1 = new Thread(() -> r1.method1(r2), "Thread-1");
        Thread t2 = new Thread(() -> r2.method1(r1), "Thread-2");

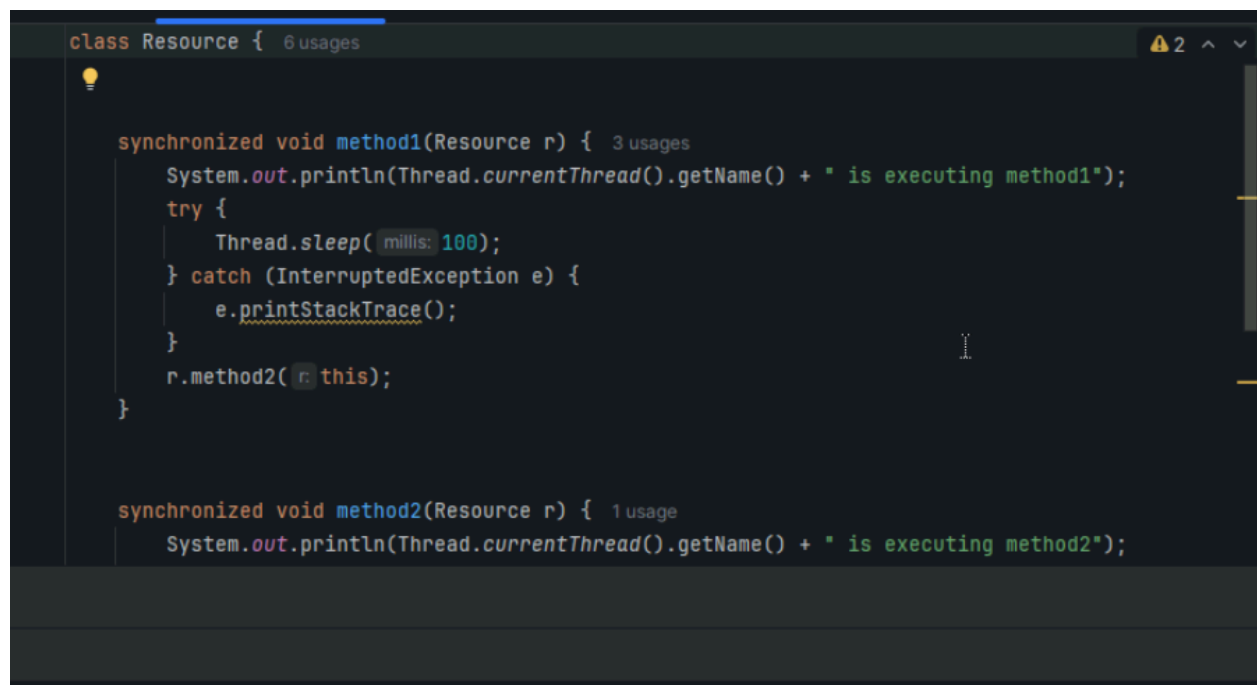
        t1.start();
        t2.start();
    }
}

```



```
1 public class DeadlockExample {
2     public static void main(String[] args) {
3         final Resource r1 = new Resource();
4         final Resource r2 = new Resource();
5
6         Thread t1 = new Thread(() -> r1.method1(r2), name: "t1");
7
8         Thread t2 = new Thread(() -> r2.method1(r1), name: "t2");
9
10        t1.start();
11        t2.start();
12    }
13 }
14
15
```

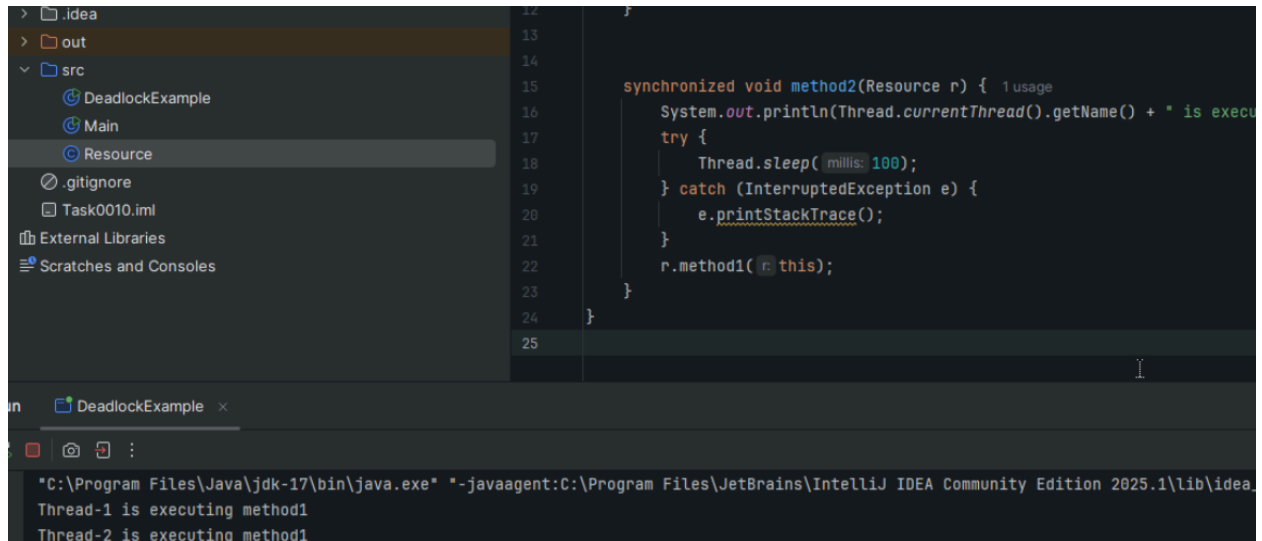
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2023.1\lib\idea_rt.jar=1273.1273:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2023.1\bin\java.exe" [Debug]
Thread-1 is executing method1
Thread-2 is executing method1



```
class Resource { 6 usages

    synchronized void method1(Resource r) { 3 usages
        System.out.println(Thread.currentThread().getName() + " is executing method1");
        try {
            Thread.sleep(millis: 100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        r.method2(r, this);
    }

    synchronized void method2(Resource r) { 1 usage
        System.out.println(Thread.currentThread().getName() + " is executing method2");
    }
}
```



Task 11:

Inter- thread communication...

Example of Inter-thread Communication

```
class SharedResource {  
    private boolean ready = false;  
  
    synchronized void produce() {  
        try {  
            while (ready) {  
                wait();  
            }  
            System.out.println("Producing...");  
            ready = true;  
            notify();  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
  
    synchronized void consume() {  
        try {
```

```

        while (!ready) {
            wait();
        }
        System.out.println("Consuming...");
        ready = false;
        notify();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

```

```

public class InterThreadCommunicationExample {
    public static void main(String[] args) {
        SharedResource resource = new SharedResource();

        Thread producer = new Thread(resource::produce);
        Thread consumer = new Thread(resource::consume);

        producer.start();
        consumer.start();
    }
}

```

```

> public class InterThreadCommunicationExample {
>     public static void main(String[] args) {
        SharedResource resource = new SharedResource();

        // Producer thread
        Thread producer = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                resource.produce();
            }
        });

        // Consumer thread
        Thread consumer = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                resource.consume();
            }
        });
    }
}

```

```

.gitignore
Task0011.iml
ernal Libraries
atches and Consoles

17
18
19
20
21
22
23
    });
    producer.start();
    consumer.start();
}

InterThreadCommunicationExample x
...
suming...
ducing...
suming...
...
cess finished with exit code 0

```

Task 12:

Interrupting a thread

Example of Interrupting a Thread

```

class InterruptibleThread extends Thread {
    public void run() {
        try {
            while (!Thread.currentThread().isInterrupted()) {
                System.out.println("Thread is running");
                Thread.sleep(100);
            }
        } catch (InterruptedException e) {
            System.out.println("Thread was interrupted");
        }
    }
}

```

```

public class InterruptExample {
    public static void main(String[] args) {
        InterruptibleThread thread = new InterruptibleThread();
        thread.start();

        try {
            Thread.sleep(500);
            thread.interrupt();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

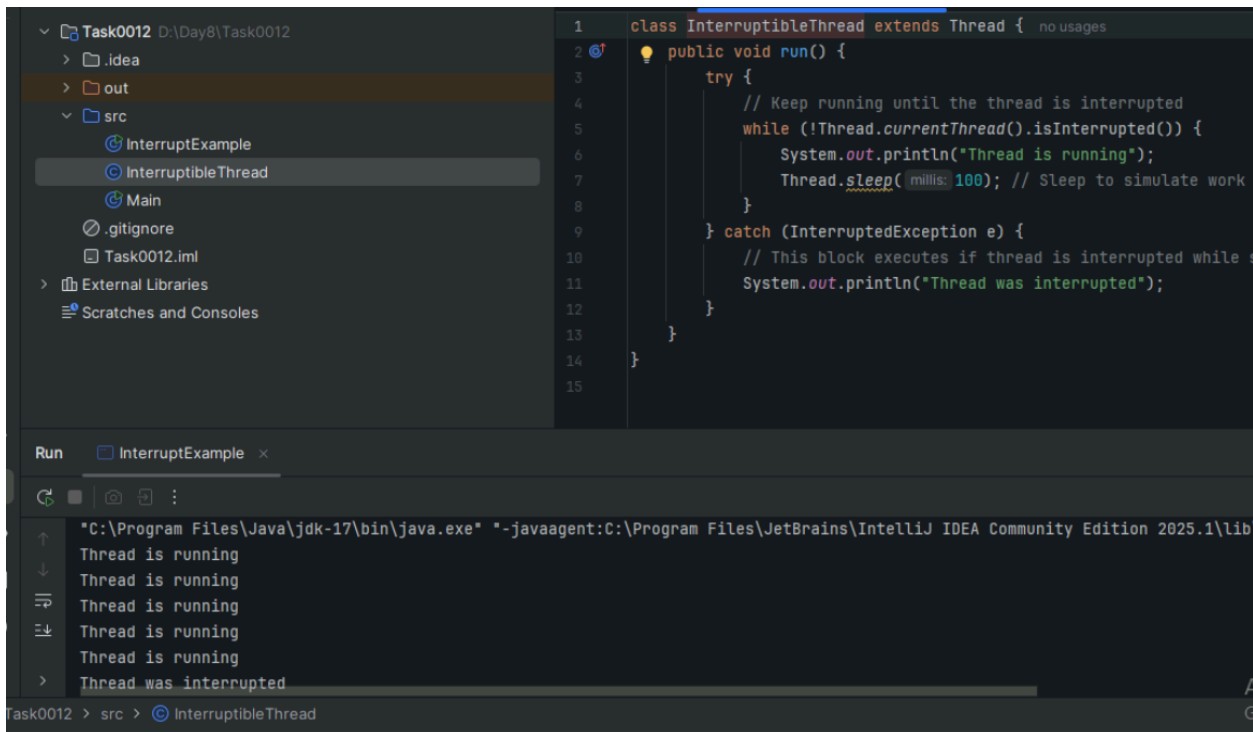
Task 10 to task 12 - 5 min each..

1.12 to 1.28

.. plz finish the tasks ... and raise your hands..

```
public class InterruptExample {
    public static void main(String[] args) {
        InterruptibleThread thread = new InterruptibleThread();
        thread.start(); // Start the thread

        try {
            Thread.sleep(500); // Let the thread run for a bit
            thread.interrupt(); // Interrupt the thread after 500ms
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```



Task 14::

What is Thread pool?

A thread pool is a collection (or pool) of pre-instantiated, reusable threads that are ready to perform tasks. Instead of creating a new thread every time a task needs to run—which can be expensive in terms of system resources—a thread pool manages a fixed number of threads and assigns tasks to these threads.

Task 15:

Run the below code and see the file with the given name created or not..
Run it again with I like India instead of I love India.. And see the file ...

```
public class WriteByte
{
    public static void main(String args[])
    {
        File f1=new File("FileName01.txt"); \\ to
        create new file FileOutputStream outfile =
        null;
        byte Text[] = {'I',' ','L','O','V','E',' ','T','N','D','T','A'};
        try
        {
            outfile = new FileOutputStream(f1);
            outfile.write(Text);
        }
        catch(IOException e)
        {
            System.out.println(e);
            System.exit(-1);
        }
        System.out.println("Write Byte");
        System.out.println("Thank You...!!!");
    }
}
```

}

```
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;

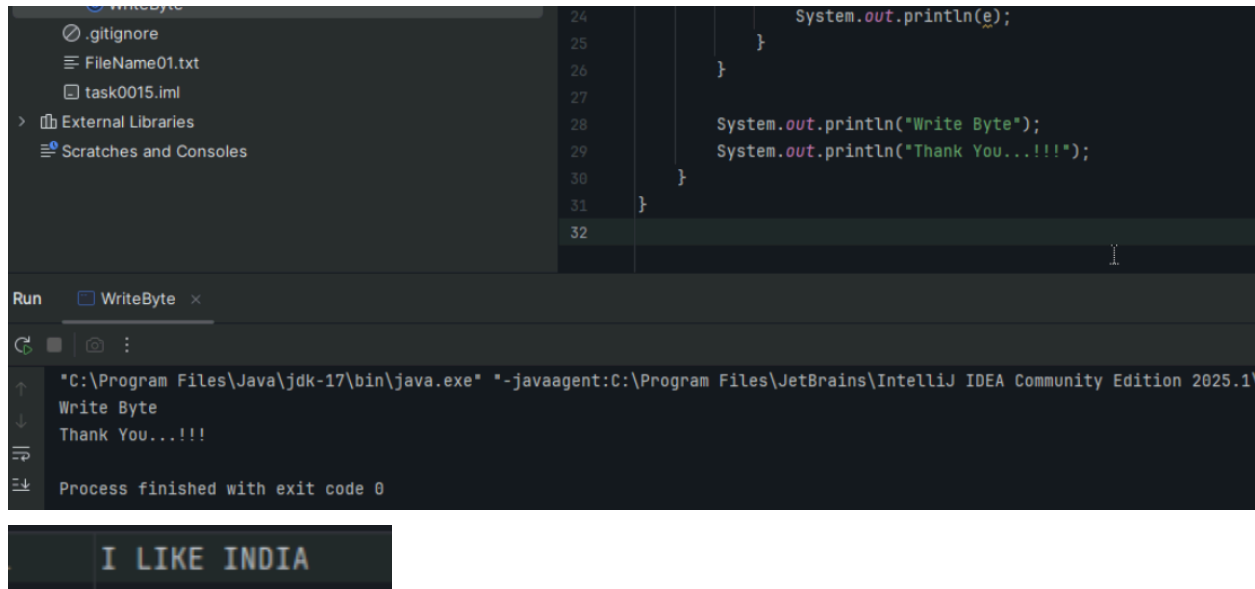
> public class WriteByte {
>     public static void main(String args[]) {
        File f1 = new File( pathname: "FileName01.txt"); // Create a new file
        FileOutputStream outfile = null;

        // Correct byte array for "I LOVE INDIA"
        byte[] Text = {'I', ' ', 'L', 'O', 'V', 'E', ' ', 'I', 'N', 'D', 'I', 'A'};

        try {
            outfile = new FileOutputStream(f1); // Create and open file
            outfile.write(Text);                // Write bytes
        } catch (IOException e) {
```

```
        } catch (IOException e) {
            System.out.println(e);
            System.exit( status: -1);
        } finally {
            try {
                if (outfile != null)
                    outfile.close(); // Close the stream
            } catch (IOException e) {
                System.out.println(e);
            }
        }

        System.out.println("Write Byte");
        System.out.println("Thank You...!!!");
```



Task 16:

Try this code to see the output ...

Write a program which reads byte from file.

```
import java.io.*;
public class ReadingByte
{
    public static void main(String args[])
    {
        FileInputStream infile = null;
        int b;
        try
        {
            infile = new FileInputStream("FileName01.txt");
            while((b = infile.read()) != -1)
            {
                System.out.println((char)b);
            }
            infile.close();
        }
        catch(IOException e)
        {
        }
```

```
        System.out.println("Sorry..!! File Not Found...!!!");
    }
}
}
```

```
import java.io.*;
public class ReadingByte
{
    public static void main(String args[])
    {
        FileInputStream infile = null;
        int b;
        try
        {
            infile = new FileInputStream( name: "FileName01.txt");
            while((b = infile.read()) != -1)
            {
                System.out.println((char)b);
            }
            infile.close();
        }
    }
}

4      public static void main(String args[])
11      while((b = infile.read()) != -1)
12      {
13          System.out.println((char)b);
14      }
15      infile.close();
16  }
17  catch(IOException e)
18  {
19      System.out.println("Sorry...!! File Not Found...!!!");
20  }
21  }
22  }
23  }
```

File Explorer: .idea, out, src, Main, ReadingByte, .gitignore, Task0016.iml, External Libraries, Scratches and Consoles

Console Output:

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_r
Sorry...!! File Not Found...!!!

Process finished with exit code 0
```

Task 17:

Taking input from the user and writing on the file...
Create a file and see the output...

```
import java.io.*;

import java.util.*;

public class WriteByte_1

{

    public static void main(String args[]) {

        FileOutputStream outfile = null;

        //String s=args[0]; // to input string from

        command line Scanner sc=new

        Scanner(System.in);

        String s=sc.nextLine();

        byte b1[] = s.getBytes();

        try

        {

            outfile = new FileOutputStream("FileName02.txt");

            outfile.write(b1);

        }

        catch(IOException e)

        {

            System.out.println(e);

            System.exit(-1);

        }

        System.out.println("Write Byte");

        System.out.println("Thank You...!!!");

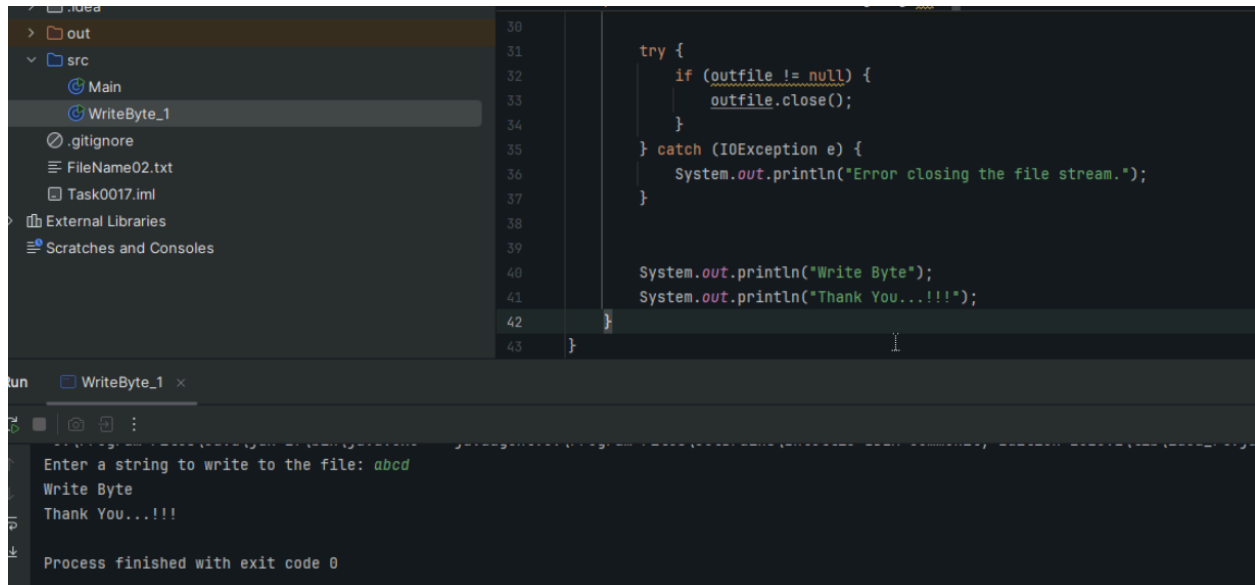
    }

}
```

```
}  
  
}
```

```
1  import java.io.*;  
2  import java.util.*;  
3  
4  public class WriteByte_1 {  
5      public static void main(String args[]) {  
6          FileOutputStream outfile = null;  
7  
8  
9          Scanner sc = new Scanner(System.in);  
10         System.out.print("Enter a string to write to the file: ");  
11  
12  
13         String s = sc.nextLine();  
14  
15  
16         byte b1[] = s.getBytes();
```

```
        try {  
            outfile = new FileOutputStream("FileName02.txt");  
            outfile.write(b1);  
        } catch (IOException e) {  
            System.out.println(e);  
            System.exit(-1);  
        }  
  
    }  
}
```



The screenshot shows an IDE with a project structure on the left, a code editor in the center, and a console window at the bottom. The project structure includes a 'src' folder with 'Main' and 'WriteByte_1' files. The code editor shows a Java program that attempts to close a file stream, catches an exception, and prints messages. The console window shows the program's execution, including user input and the final exit code.

```
30
31
32     try {
33         if (outfile != null) {
34             outfile.close();
35         }
36     } catch (IOException e) {
37         System.out.println("Error closing the file stream.");
38     }
39
40     System.out.println("Write Byte");
41     System.out.println("Thank You...!!!");
42 }
43 }
```

Run WriteByte_1 x

Enter a string to write to the file: *abcd*
Write Byte
Thank You...!!!
Process finished with exit code 0

Task 18:

Write a program which creates file and writes character into that file.

```
import java.io.*;
Class CharacterWrite {
    public static void main(String args[]) {
        File f1=new File("FileName03.txt");
        FileWriterfw = null;
        try {
            fw=new FileWriter(f1);
            fw.write("ahmedabad \n");
            fw.write(" baroda \n");
            fw.close();
        }
        catch(FileNotFoundException e)
        {
            System.out.println("Sorry..!! File Not Found...!!!");
        }
        catch(IOException e)
        {

```

```
        System.out.println(e.getMessage());
    }
    System.out.println(" write operation done!!");
}
}
```

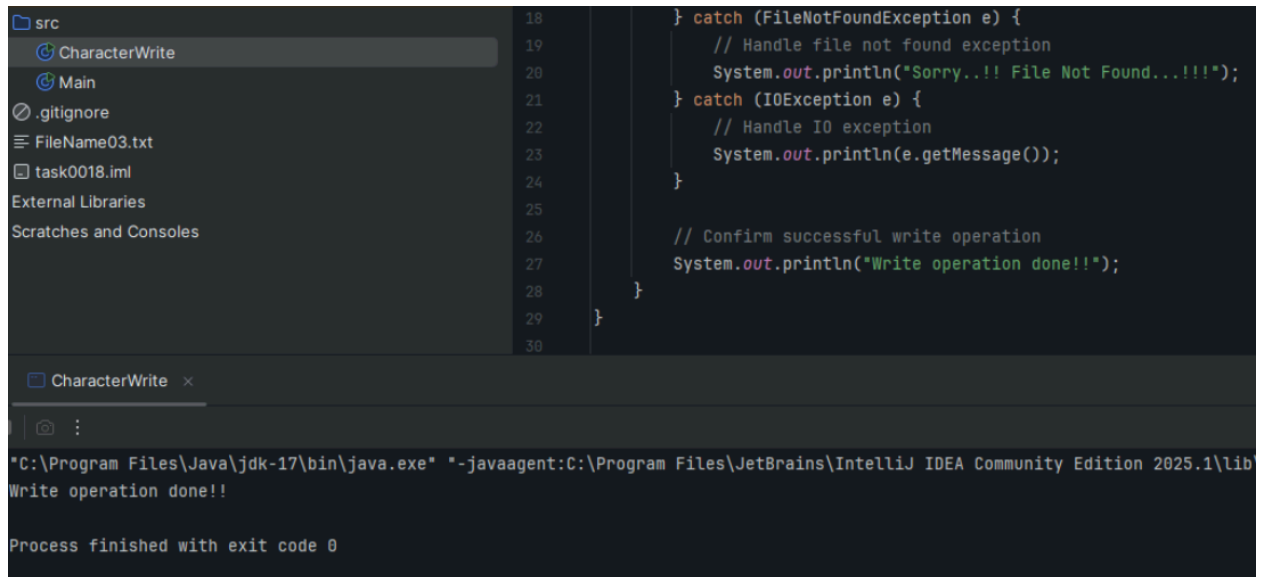
```
import java.io.*;

public class CharacterWrite {
    public static void main(String args[]) {
        File f1 = new File("FileName03.txt");
        FileWriter fw = null;

        try {
            // Create a FileWriter object to write to the file
            fw = new FileWriter(f1);

            // Write data to the file
            fw.write("Meera \n");
            fw.write("J \n");

            // Close the FileWriter to release resources
```



```
src
  CharacterWrite
  Main
.gitignore
FileName03.txt
task0018.iml
External Libraries
Scratches and Consoles

18      } catch (FileNotFoundException e) {
19          // Handle file not found exception
20          System.out.println("Sorry...!! File Not Found...!!!");
21      } catch (IOException e) {
22          // Handle IO exception
23          System.out.println(e.getMessage());
24      }
25
26      // Confirm successful write operation
27      System.out.println("Write operation done!!");
28  }
29  }
30

CharacterWrite x
Write operation done!!

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib
Process finished with exit code 0
```

Task 19

Write a program which reads character from file.

```
import java.io.*;
```

Class Readchar

```
{
    public static void main(String args[])
    {
        FileReader fr = null;
        try
        {
            fr = new FileReader("FileName03.txt");
            int ch;
            while((ch = fr.read()) != -1)
            {
                System.out.print((char)ch);
            }
            System.out.println("Reading complete");
            fr.close();
        }
    }
}
```

```

        catch(FileNotFoundException e)
        {
            System.out.println("Sorry..!! File Not Found...!!!");
        }
        catch(IOException e)
        {
            System.out.println(e.getMessage());
        }
    }
}

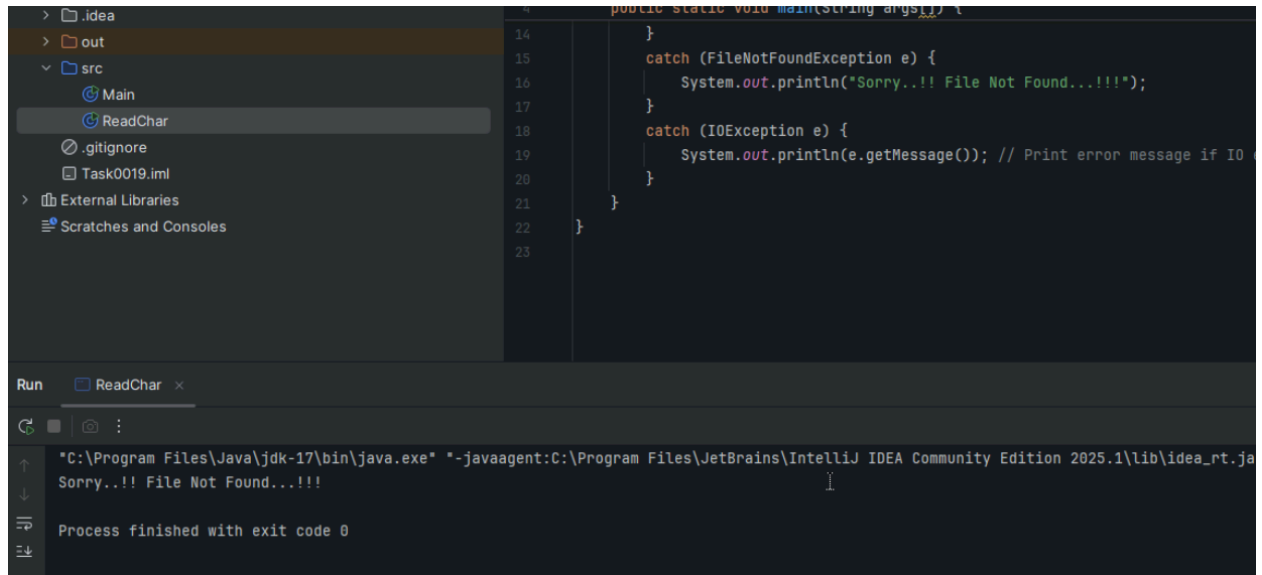
```

```

import java.io.*;

public class ReadChar {
    public static void main(String args[]) {
        FileReader fr = null;
        try {
            fr = new FileReader("FileName03.txt"); // Open the file for reading
            int ch; // To store the character read
            while ((ch = fr.read()) != -1) {
                System.out.print((char) ch); // Print the character read
            }
            System.out.println("\nReading complete"); // Indicate reading is done
            fr.close(); // Close the file reader
        }
        catch (FileNotFoundException e) {
            System.out.println("Sorry..!! File Not Found...!!!");
        }
    }
}

```



Task 20

Write a program to read one byte at a time from a file and copy it into another file immediately.

```
import java.io.*;  
Class CopyByte  
{  
    public static void main(String args[])  
    {  
        try  
        {  
            byte b=0;  
            FileInputStream infile = new FileInputStream("NewFile01.txt");  
            FileOutputStream outfile = new FileOutputStream("NewFile05.txt");  
            while(byteread != -1)  
            {  
                b = (byte)infile.read();  
                outfile.write(b);  
            }  
            System.out.println("Byte Copied From in.txt to out.txt File ");  
        }  
        catch(FileNotFoundException e)
```

```

        {
            System.out.println("Sorry..!! File Not Found...!!!");
        }
        catch(IOException e)
        {
            System.out.println(e.getMessage());
        }
    }
}

```

```

import java.io.*;

class CopyByte {
    public static void main(String args[]) {
        try {
            byte b = 0; // Declare a byte variable to store each byte read
            FileInputStream infile = new FileInputStream("NewFile01.txt"); // Input file
            FileOutputStream outfile = new FileOutputStream("NewFile05.txt"); // Output file

            // Read one byte at a time from input file and write it to output file
            while ((b = (byte) infile.read()) != -1) { // Read byte by byte
                outfile.write(b); // Write the byte to the output file
            }

            System.out.println("Byte Copied From NewFile01.txt to NewFile05.txt File");
        } catch (FileNotFoundException e) {

```

The screenshot shows the IntelliJ IDEA interface. On the left, the Project Explorer displays the file structure with 'CopyByte' selected. The main editor shows the code for 'CopyByte.java'. The code includes file handling logic with try-catch blocks for 'FileNotFoundException' and 'IOException'. The console at the bottom shows the command to run the program and the output: 'Byte Copied From NewFile01.txt to NewFile05.txt File' and 'Process finished with exit code 0'.

```
12         outfile.write(b); // Write the byte to the output file
13     }
14
15     System.out.println("Byte Copied From NewFile01.txt to NewFile05.txt File");
16 } catch (FileNotFoundException e) {
17     System.out.println("Sorry...!! File Not Found...!!!");
18 } catch (IOException e) {
19     System.out.println(e.getMessage()); // Handle IO exceptions
20 }
21
22
23
```

CopyByte x

C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea-agent-2\agent.jar" -Dfile.encoding=UTF-8 -cp C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\bin\idea-ic-2025.1.jar C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\bin\idea-ic-2025.1.jar

Byte Copied From NewFile01.txt to NewFile05.txt File

Process finished with exit code 0

The screenshot shows the IntelliJ IDEA interface with the 'NewFile01.txt' file open. The file contains the text 'hiiiihihi' on the first line. A lightbulb icon is visible below the text, indicating a suggestion or hint.

1 hiiiihihi

The screenshot shows the IntelliJ IDEA interface with the 'NewFile05.txt' file open. The file contains the text 'hiiiihihi' on the first line. A lightbulb icon is visible below the text, indicating a suggestion or hint.

1 hiiiihihi

Task 21:

Merging two files to 3rd file..

Write a program to merge two files in third file.

```
import java.io.*;

class FileMergeDemo
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream file1 = new
            FileInputStream("NewFile01.txt");
            FileInputStream file2 = new
            FileInputStream("NewFile02.txt");
            SequenceInputStream file3 = new
            SequenceInputStream(file1, file2);
            BufferedInputStream br1 = new
            BufferedInputStream(file3);
            BufferedOutputStream br2 = new
            BufferedOutputStream(System.out);
            int ch;
            while((ch = br1.read()) != -1)
            {
                br2.write((char)ch);
            }
            br1.close();
            br2.close();
            file1.close();
            file2.close();
            System.out.println("Merge Two File Successfully ");
        }
        catch(IOException e)
```

```

    {
        System.out.println("Sorry...!! File Not Found...!!!");
    }
}

```

```

import java.io.*;

class FileMergeDemo {
    public static void main(String args[]) {
        try {

            FileInputStream file1 = new FileInputStream(name: "NewFile01.txt");
            FileInputStream file2 = new FileInputStream(name: "NewFile02.txt");

            SequenceInputStream file3 = new SequenceInputStream(file1, file2);

            BufferedInputStream br1 = new BufferedInputStream(file3);

```

```

            FileOutputStream fileOut = new FileOutputStream(name: "MergedFile.txt");

            // Create BufferedOutputStream for efficient writing to the file
            BufferedOutputStream br2 = new BufferedOutputStream(fileOut);

            int ch;
            // Read content from br1 (merged input streams) and write it to the new file
            while ((ch = br1.read()) != -1) {
                br2.write(ch);
            }

            // Close all streams to release resources
            br1.close();

```

```
int ch;
// Read content from br1 (merged input streams) and write it to the new fi
while ((ch = br1.read()) != -1) {
    br2.write(ch);
}

// Close all streams to release resources
br1.close();
br2.close();
file1.close();
file2.close();
fileOut.close();

// Print success message
```

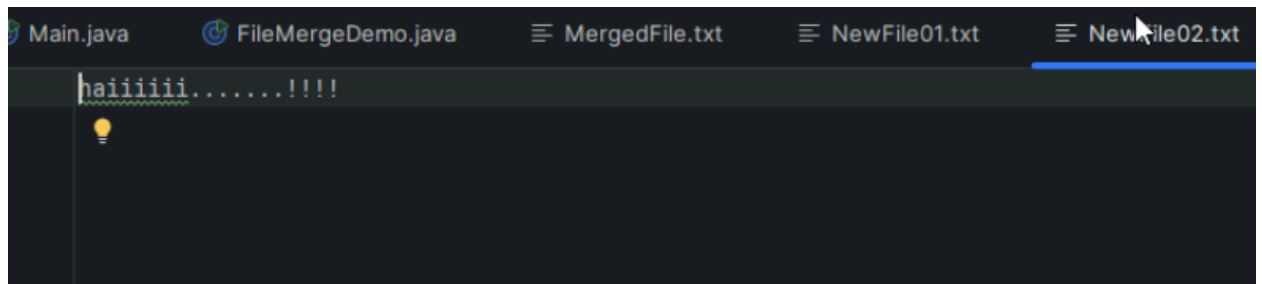
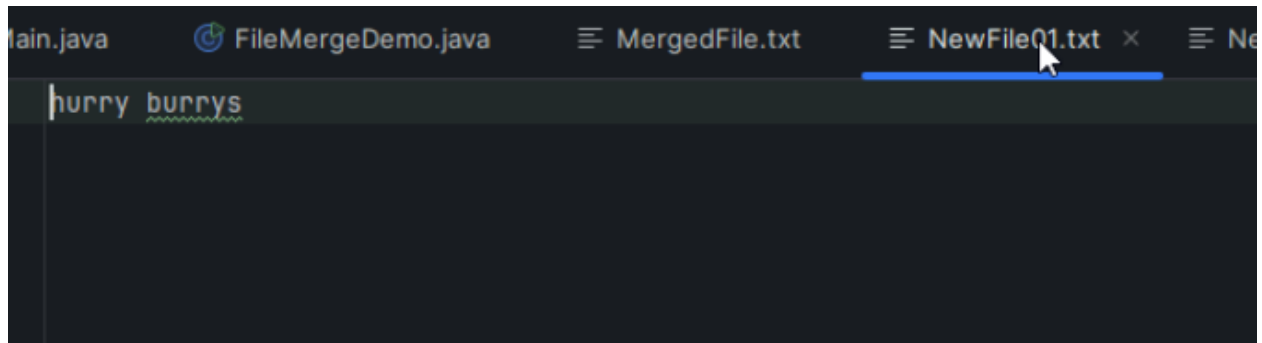
```
30      br2.close();
31      file1.close();
32      file2.close();
33      fileOut.close();
34
35      // Print success message
36      System.out.println("Files Merged Successfully into 'MergedFile.txt'");
37    } catch (IOException e) {
38      System.out.println("Sorry...!! File Not Found...!!!");
39    }
40  }
41 }
42
```

MergeDemo x

```
ram Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea
Merged Successfully into 'MergedFile.txt'
```

finished with exit code 0

```
1  hurry burryshaiiiii.....!!!!
```



Task 22:

Write an application to rename a file. Use the renameTo() method of File to accomplish

/*this task. The first command line argument is the old filename and the second is the newfilename.

*/

```
import java.io.*;
```

```
classFileRenameDemo
```

```
{
```

```
    public static void main(String args[])
```

```

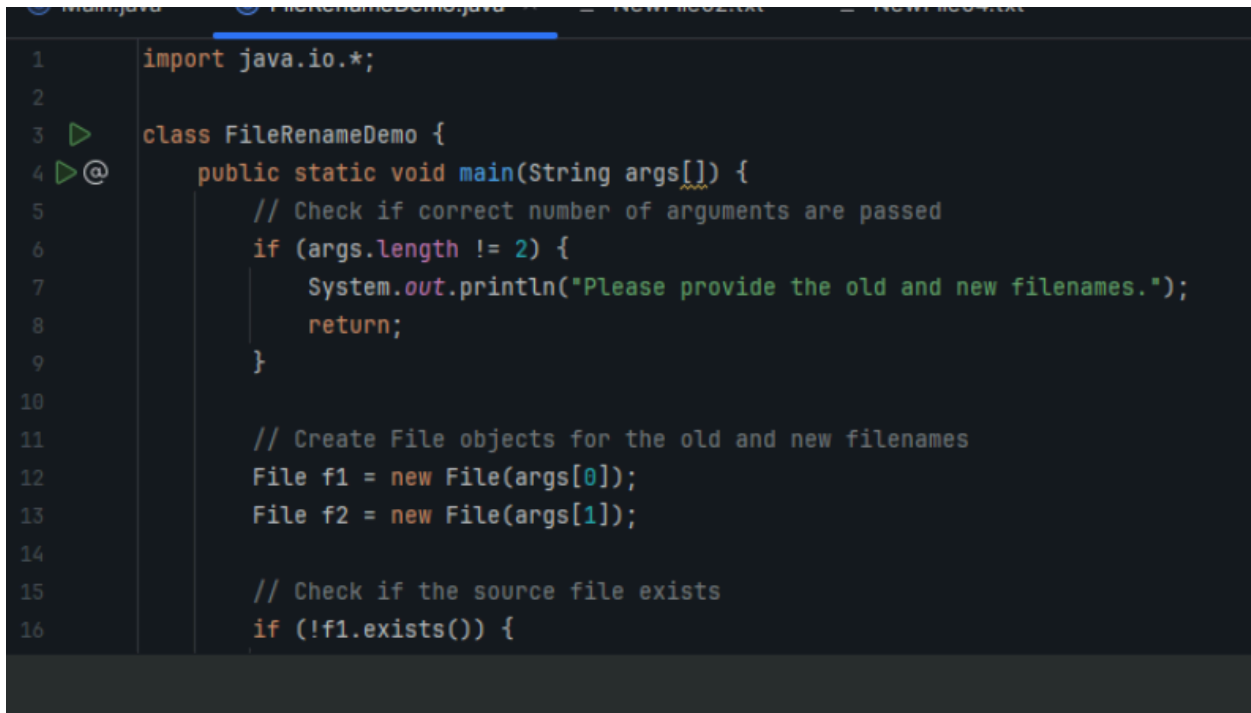
    {
        File f1 = new File(args[0]);
        File f2 = new File(args[1]);
        f1.renameTo(f2);
        System.out.println("Rename File " +f1+" To "+f2+"
        Sucessfully "); }
    }

```

Output :

Javac FileRenameDemo.java

Java FileRenameDemo input1.txt abc.txt



```

1  import java.io.*;
2
3  class FileRenameDemo {
4  public static void main(String args[]) {
5      // Check if correct number of arguments are passed
6      if (args.length != 2) {
7          System.out.println("Please provide the old and new filenames.");
8          return;
9      }
10
11     // Create File objects for the old and new filenames
12     File f1 = new File(args[0]);
13     File f2 = new File(args[1]);
14
15     // Check if the source file exists
16     if (!f1.exists()) {

```

```

// Check if the source file exists
if (!f1.exists()) {
    System.out.println("The file " + f1 + " does not exist.");
    return;
}

// Attempt to rename the file
if (f1.renameTo(f2)) {
    System.out.println("File renamed successfully: " + f1 + " to " + f2);
} else {
    System.out.println("Failed to rename file " + f1 + " to " + f2);
}
}
}

```



```

Run FileRenameDemo x
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBr
File renamed successfully: NewFile01.txt to NewFile04.txt
Process finished with exit code 0

```

Task 23 👍

Task 23 and 25 - Buffered reader and writer — for large files to be read.

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadFileExample {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new
FileReader("largefile.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        }
    }
}

```

```

    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

The screenshot shows an IDE with a Java file named `ReadFileExample.java`. The code uses `BufferedReader` to read a file line by line. The output window shows the contents of the file being read.

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadFileExample {
    public static void main(String[] args) {
        // Try-with-resources to automatically close resources after usage
        try (BufferedReader br = new BufferedReader(new FileReader("largefile.txt"))) {
            String line;
            // Read the file line by line
            while ((line = br.readLine()) != null) {
                // Print each line to the console
                System.out.println(line);
            }
        } catch (IOException e) {
            // If there is an error reading the file, print the stack trace
        }
    }
}

```

The output window shows the following text:

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=51002" -Dfile.encoding=UTF-8
Line 1: Hello, world!
Line 2: This is a sample file.
Line 3: The BufferedReader will read this file line by line.
Line 4: Let's see how it handles larger files.

```

Task 24 non buffered

```

import java.io.FileReader;
import java.io.IOException;

public class NonBufferedReaderExample {

```

```

public static void main(String[] args) {
    try (FileReader fr = new FileReader("largefile.txt")) {
        int ch;
        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

```

import java.io.FileReader;
import java.io.IOException;

public class NonBufferedReaderExample {
    public static void main(String[] args) {
        // Try-with-resources statement to handle FileReader
        try (FileReader fr = new FileReader("largefile.txt")) {
            int ch;
            // Read the file one character at a time
            while ((ch = fr.read()) != -1) {
                // Print the character to the console
                System.out.print((char) ch);
            }
        } catch (IOException e) {
            // Catch and print any IOExceptions that may occur
            e.printStackTrace();
        }
    }
}

```

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=51071" -Dfile.encoding=UTF-8

```

Hello, this is a test file for reading.
It contains multiple lines.
We will read the file character by character.

Process finished with exit code 0

```

Task 25:

Buffered Writer

```
import java.io. BufferedWriter;
import java.io. FileWriter;
import java.io. IOException;

public class BufferedWriterDemo {
    public static void main(String[] args) {
        String filePath = "example.txt";
        String content = "Hello, World!\nThis is a BufferedWriter
example.";

        // Initialize BufferedWriter with a FileWriter
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filePath))) {
            // Write content to the file
            writer.write(content);
            System.out.println("Content written to file.");
        } catch (IOException e) {
            System.err.println("An error occurred: " + e.getMessage());
        }
    }
}
```

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class BufferedReaderExample {
    public static void main(String[] args) {
        String filePath = "example.txt"; // Path where the file will be created
        String content = "Hello, World!\nThis is a BufferedWriter example."; // Content to write

        // Initialize BufferedWriter with a FileWriter
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
            // Write content to the file
            writer.write(content);
            System.out.println("Content written to file.");
        } catch (IOException e) {
            System.err.println("An error occurred: " + e.getMessage());
        }
    }
}
```

The screenshot shows the IntelliJ IDEA interface. On the left, the 'Project' tool window displays the project structure, including the 'Main' class and the 'example.txt' file. The 'Main' class is selected, and its code is visible in the editor. The console at the bottom shows the output of the program, which is 'Content written to file.'.

```
12 // Write content to the file
13 writer.write(content);
14 System.out.println("Content written to file.");
15 } catch (IOException e) {
16     System.err.println("An error occurred: " + e.getMessage());
17 }
18 }
19 }
20 }
```

BufferedReaderExample x

"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1\lib\idea_rt.jar=5132:C:\Program Files\Java\jdk-17\bin" -Dfile.encoding=UTF-8

Content written to file.

Process finished with exit code 0

The screenshot shows the IntelliJ IDEA interface with the 'example.txt' file open in the editor. The file contains two lines of text: 'Hello, World!' and 'This is a BufferedWriter example.'.

```
1 Hello, World!
2 This is a BufferedWriter example.
```